Supplementary Material for SE-RGCL: Robust Graph Contrastive Learning against Adversarial Traffic Attacks in IoV

Zhong Li, Yu Dong, Huiying Huang, Mingjian Guang

APPENDIX A
SE-RGCL HYPERPARAMETER SETTINGS

Table I summarizes the complete hyperparameter configuration used in SE-RGCL. The parameter settings are kept fixed across the main experiments unless otherwise specified. The memory-bank capacities and hard negative mining configuration are selected to balance representation diversity and computational cost. The LLR regularization parameter follows the setting adopted in [9], while the remaining hyperparameters are selected according to validation performance.

The STM and LTM capacities are set to 512 and 2048, respectively, with an 8:2 LTM-to-STM sampling ratio to preserve long-term representation diversity while retaining recent batch information. The top $M = 128$ negatives with the highest cosine similarity to each anchor are used for hard negative mining. The EMA coefficient $\eta = 0.9$ stabilizes the long-term statistics across batches, and the temperature parameter is set to $\tau = 0.1$. The final objective uses $\epsilon_1 = 1.0$, $\epsilon_2 = 0.5$, and $\epsilon_3 = 0.1$ for the contrastive, prototype-alignment, and orthogonality terms, respectively.

TABLE I. Hyperparameter Settings of SE-RGCL.

Parameter	**Value**
GAT layers	2
Attention heads	8 / 1
Hidden dimension	512
Pre-training epochs T	100
Batch size $\|B\|$	256
STM capacity S	512
LTM capacity L	2048
LTM:STM sampling ratio	8:2
Hard negatives M	128
LLR regularization λ	0.001
EMA coefficient η	0.9
Temperature τ	0.1
Loss weight ϵ_1	1.0
Loss weight ϵ_2	0.5
Loss weight ϵ_3	0.1

APPENDIX B
CLEAN-DATA PERFORMANCE EVALUATION

We compare SE-RGCL with a wide range of GNN baselines, from foundational architectures (GCN [1], GAT [2], GIN [3], GraphSAGE [4]) to more advanced models that handle deeper propagation (GCNII [5]), improve robustness (Grand [6]), and aggregate higher-order information (MixHop [7], H_2GCN [8]). The evaluation of all models is conducted on the NSL-KDD and UNSW-NB15 datasets without adding any data perturbation (*i.e.*, no adversarial attacks are implemented) to accurately measure their inherent performance under ideal conditions. As detailed in Table II and Table III, SE-RGCL consistently achieves strong performance across the main evaluation metrics.

TABLE II
Model Performance Evaluation on the NSL-KDD Dataset.

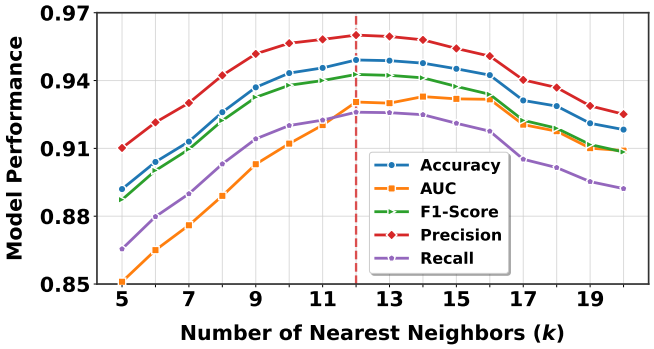
Models	Acc	Precision	Recall	F1 score	AUC
GCN	0.7650	0.9210	0.6430	0.7570	0.9150
GAT	0.7440	0.9090	0.6700	0.7710	0.7900
GIN	0.8060	0.9530	0.6940	0.8030	0.8240
GraphSAGE	0.8000	0.9650	0.6730	0.7930	0.8200
GCNII	0.8550	0.9701	0.7850	0.8651	0.9335
Grand	0.9091	0.9750	0.8010	0.8821	0.9512
MixHop	0.8750	0.9715	0.8552	0.9105	0.9400
H_2GCN	0.9527	0.9726	0.8234	0.8500	0.9601
SE-RGCL	**0.9821**	**0.9824**	**0.9818**	**0.9820**	**0.9820**

TABLE III
Model Performance Evaluation on the UNSW-NB15 Dataset.

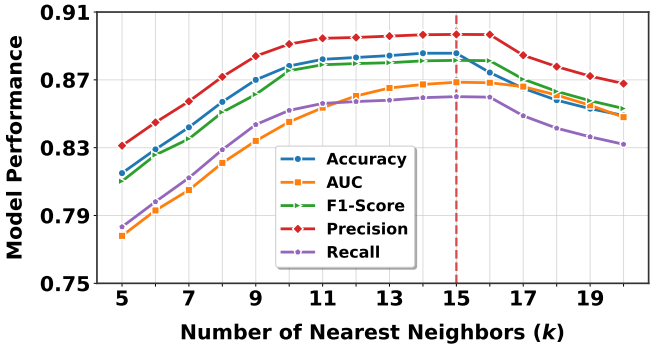
Models	Acc	Precision	Recall	F1 score	AUC
GCN	0.8446	0.8404	0.8395	0.8399	0.8904
GAT	0.9131	0.9101	0.9002	0.9047	0.9104
GIN	0.9154	0.9091	0.9071	0.9081	0.9015
GraphSAGE	0.9193	0.9167	0.9072	0.9116	0.9132
GCNII	0.9225	0.9280	0.9135	0.9205	0.9195
Grand	0.9310	0.9355	0.9108	0.9270	0.9260
MixHop	0.9216	0.9205	0.9255	0.9228	0.9295
H_2GCN	0.9132	0.9085	0.9211	0.9013	0.8926
SE-RGCL	**0.9477**	**0.9538**	**0.9289**	**0.9398**	**0.9336**

APPENDIX C
ADDITIONAL ANALYSIS OF NEIGHBORHOOD SIZE k

To further validate the structural entropy-guided neighborhood selection, we evaluate the IDS performance under different candidate values of k on NSL-KDD and UNSW-NB15. As shown in Fig. 1, the performance remains relatively stable around the neighborhood sizes selected by 1-DSE. For NSL-KDD, the highest performance is observed around $k = 12$, while the selected $k = 11$ lies within the corresponding high-performance region. For UNSW-NB15, the selected $k = 14$ is close to the empirical performance peak at $k = 15$. These results further support the use of 1-DSE for determining an appropriate neighborhood size for graph construction.



(a) Results on NSL-KDD dataset



(b) Results on UNSW-NB15 dataset

Fig. 1. Performance analysis of the IDS model with varying neighbor number k.

Appendix D
Performance Stability Analysis

To evaluate the stability of SE-RGCL, all experiments are repeated over five independent runs with different random seeds while keeping the data partitions and hyperparameters fixed.

We report the mean and standard deviation of F1 score and AUC under clean conditions and three representative white-box adversarial attacks, including FGSM, MIM, and CW. FGSM represents an attack involved in representation learning, MIM evaluates robustness to an unseen iterative attack, and CW provides a stronger optimization-based adversarial setting. As shown in Table IV, SE-RGCL maintains consistently high mean performance with small standard deviations across both NSL-KDD and UNSW-NB15. The limited inter-run variation indicates that the robustness gains of SE-RGCL remain stable under different random initializations and stochastic training processes.

Appendix E
Sensitivity Analysis of Mobility-Aware Graph Parameters

We further analyze the sensitivity of SE-RGCL to the time-window length Δt, V2V communication radius R_{V2V}, and RSU communication radius R_{RSU} using a one-factor-at-a-time strategy. As shown in Table V, overly short or long time windows reduce performance due to insufficient temporal aggregation or weakened mobility consistency, respectively. Similarly, excessively small communication radii restrict feasible candidate connections, whereas overly large radii introduce less relevant node relationships. The settings $\Delta t = 10$s, $R_{\mathrm{V2V}} = 200$m, and $R_{\mathrm{RSU}} = 500$m provide the best overall performance on both datasets. The results demonstrate that SE-RGCL maintains stable performance over moderate parameter ranges and benefits from balanced temporal and communication constraints during graph construction.

TABLE IV. Performance Stability of SE-RGCL

Dataset	Setting	F1	AUC
NSL-KDD	Clean	0.9813 ± 0.0016	0.9820 ± 0.0014
	FGSM	0.9770 ± 0.0021	0.9765 ± 0.0018
	MIM	0.9775 ± 0.0023	0.9773 ± 0.0020
	CW	0.9064 ± 0.0047	0.9212 ± 0.0041
UNSW-NB15	Clean	0.9332 ± 0.0028	0.9450 ± 0.0025
	FGSM	0.9310 ± 0.0032	0.9440 ± 0.0029
	MIM	0.9307 ± 0.0035	0.9324 ± 0.0031
	CW	0.8890 ± 0.0056	0.8903 ± 0.0051

TABLE V. Sensitivity Analysis of Mobility-Aware Graph Parameters.

Parameter	Value	NSL-KDD		UNSW-NB15	
		F1	AUC	F1	AUC
Δt	5 s	0.9684	0.9701	0.9192	0.9236
	10 s	**0.9778**	**0.9776**	**0.9311**	**0.9320**
	15 s	0.9721	0.9734	0.9254	0.9271
	20 s	0.9647	0.9662	0.9168	0.9203
R_{V2V}	100 m	0.9689	0.9707	0.9201	0.9239
	200 m	**0.9778**	**0.9776**	**0.9311**	**0.9320**
	300 m	0.9734	0.9741	0.9276	0.9290
	400 m	0.9658	0.9673	0.9187	0.9214
R_{RSU}	300 m	0.9697	0.9712	0.9215	0.9247
	500 m	**0.9778**	**0.9776**	**0.9311**	**0.9320**
	750 m	0.9738	0.9746	0.9279	0.9293
	1000 m	0.9664	0.9681	0.9196	0.9221

Appendix F
Baseline Implementation and Hyperparameter Settings

For a fair comparison, all baseline methods are re-trained using the same training, validation, and test partitions, network traffic preprocessing procedures, and adversarial threat model as SE-RGCL. Model architectures and hyperparameters explicitly reported in the corresponding publications or official implementations are retained where applicable. Dataset-dependent or unspecified hyperparameters are selected according to validation performance, and the test set is used only for final evaluation after model and hyperparameter selection. The implementation and hyperparameter settings of all baseline methods are summarized in Table VI.

References

[1] T. N. Kipf, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.

TABLE VI. Implementation and Hyperparameter Settings of Baseline Methods.

Method	**Main Configuration**	**Key Settings / Parameter Selection**
Artifact [10]	Bayesian uncertainty estimation using MC-dropout combined with KDE-based density estimation	MC-dropout $T = 50$; dropout rate $= 0.5$; KDE applied to deep-feature representations; original detection mechanism retained.
TAD [11]	Transfer-learning-based adversarial detector with three fully connected ReLU hidden layers	Three hidden layers with 256 units per layer; detector trained for 30 epochs; unspecified dataset-dependent settings selected according to validation performance.
Omni [12]	Hyperparameter-optimized ensemble of unexpected models with weighted ensemble prediction	TPE-based optimization over activation functions, hidden-layer sizes, dropout, optimizer, batch size, epochs, and learning rate; dropout $\in [0, 0.5]$; batch size $\in \{16, 32, 64, 128\}$; epochs $\in [5, 20]$; learning rate $\in \{0.001, 0.01, 0.1\}$.
SimGRACE [13]	Encoder-perturbation-based graph contrastive-learning framework	Batch size $= 128$; pre-training epochs $= 20$; original encoder-perturbation mechanism retained; dataset-dependent learning settings selected using the validation set.
DL Ens. [14]	Deep-learning ensemble with adversarial training, data balancing, feature preprocessing and engineering, and model fine-tuning	Original ensemble and adversarial-training strategy retained; method-specific unspecified settings selected according to validation performance.
RAIDS [15]	Two autoencoders combined with multiple classifiers and LightGBM-based fusion	Reconstruction errors from two autoencoders and outputs of multiple classifiers are fused by LightGBM; original multi-feature-set and fusion strategy retained; unspecified parameters selected using the validation set.
DLL-IDS [16]	Five-hidden-layer FNN combined with an LID-based adversarial-example detector	Five hidden layers; LID neighborhood size $k = 10$; SVM-based adversarial-example detector; original LID-based detection procedure retained.
RHAD [17]	Hierarchical VAE-based anomaly detector with shallow and in-depth detection stages	VAE architecture $(41, 30, 20, 30, 41)$; learning rate $= 0.001$; batch size $= 128$; reconstruction-error screening followed by feature-impact-based in-depth detection.
Apollon [18]	Multi-classifier defense with Multi-Armed Bandit selection and Thompson sampling	Candidate classifiers include MLP, NB, RF, DT, and LR; MLP hidden size $= 32$; RF estimators $= 100$; MLP maximum iterations $= 200$; original dynamic classifier-selection strategy retained.
CNN-LSTM [19]	Distributed CNN–LSTM intrusion-detection architecture implemented under Apache Spark	Original distributed CNN–LSTM architecture retained; unspecified training parameters selected according to validation performance.
Jin-IDS [20]	OBLR-based oversampling, LOF outlier detection, metric learning, genetic-algorithm-based feature selection, and LightGBM classification	SMOTE with OBLR for resampling; LOF outlier proportion $P = 0.36$; GA crossover probability $= 0.7$, mutation probability $= 0.03$, population size $= 20$; LightGBM used as the final classifier.

[3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," *arXiv preprint arXiv:1810.00826*, 2018.

[4] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in Neural Information Processing Systems*, vol. 30, pp. 1024–1034, 2017.

[5] M. Chen, Z. Wei, Z. Huang, B. Papalexakis, and J. Ma, "Simple and deep graph convolutional networks," in *Proc. ICML*, Vienna, Austria, Jul. 2020, pp. 1725–1735.

[6] W. Feng, J. Zhang, Y. Dong, K. Wang, and J. Tang, "Graph random neural networks for semi-supervised learning on graphs," *Advances in Neural Information Processing Systems*, vol. 33, pp. 22092–22103, 2020.

[7] S. Abu-El-Haija, B. Perozzi, A. Kapoor, H. Harutyunyan, N. Alipourfard, K. Lerman, G. Ver Steeg, and A. Galstyan, "Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing," in *Proc. ICML*, Long Beach, CA, USA, Jun. 2019, pp. 21–29.

[8] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," *Advances in Neural Information Processing Systems*, vol. 33, pp. 7793–7804, 2020.

[9] E. J. Candès, X. Li, Y. Ma, and J. Wright, "Robust principal component analysis?," *Journal of the ACM*, vol. 58, no. 3, pp. 1–37, 2011.

[10] R. Feinman, R. R. Curtin, S. Shintre, and A. B. Gardner, "Detecting adversarial samples from artifacts," *arXiv preprint arXiv:1703.00410*, 2017.

[11] I. Debicha, R. Bauwens, T. Debatty, J.-M. Dricot, T. Kenaza, and W. Mees, "TAD: Transfer learning-based multi-adversarial detection of evasion attacks against network intrusion detection systems," *Future Generation Computer Systems*, vol. 138, pp. 185–197, 2023.

[12] R. Shu, T. Xia, L. Williams, and T. Menzies, "Omni: Automated ensemble with unexpected models against adversarial evasion attack," *Empirical Software Engineering*, vol. 27, Art. no. 26, 2022.

[13] J. Xia, L. Wu, J. Chen, B. Hu, and S. Z. Li, "SimGRACE: A simple framework for graph contrastive learning without data augmentation," in *Proc. ACM Web Conf. (WWW)*, Lyon, France, Apr. 2022, pp. 1070–1079.

[14] B. Tafreshian and S. Zhang, "A defensive framework against adversarial attacks on machine learning-based network intrusion detection systems," in *Proc. IEEE TrustCom*, Shanghai, China, Nov. 2024, pp. 2436–2441.

[15] A. Sarıkaya, B. G. Kılıç, and M. Demirci, "RAIDS: Robust autoencoder-based intrusion detection system model against adversarial attacks," *Computers & Security*, vol. 135, Art. no. 103483, 2023.

[16] X. Yuan, S. Han, W. Huang, Y. Liu, and J. Li, "A simple framework to enhance the adversarial robustness of deep learning-based intrusion detection system," *Computers & Security*, vol. 137, Art. no. 103644, 2024.

[17] J. Rheey and H. Park, "Robust hierarchical anomaly detection using feature impact in IoT networks," *ICT Express*, vol. 11, no. 2, pp. 358–363, 2025.

[18] A. Paya, S. Arroni, V. García-Díaz, and A. Gómez, "Apollon: A robust defense system against adversarial machine learning attacks in intrusion detection systems," *Computers & Security*, vol. 136, Art. no. 103546, 2024.

[19] A. Alferaidi, K. Yadav, Y. Alharbi, N. Razmjooy, W. Viriyasitavat, K. Gulati, S. Kautish, and G. Dhiman, "Distributed deep CNN-LSTM model for intrusion detection method in IoT-based vehicles," *Mathematical Problems in Engineering*, vol. 2022, pp. 1–8, Mar. 2022.

[20] F. Jin, M. Chen, W. Zhang, Y. Yuan, and S. Wang, "Intrusion detection on Internet of Vehicles via combining log-ratio oversampling, outlier detection and metric learning," *Information Sciences*, vol. 579, pp. 814–831, Nov. 2021.